

Adaptación de Controladores a Spring Security + JWT

Resumen del Cambio

Después del merge de `auth`, `guards` e `interceptors`, **todos los endpoints requieren autenticación JWT.**

Cambio en SecurityConfig

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/auth/**").permitAll() // ← Solo login/signup
    .anyRequest().authenticated()                // ← TODO LO DEMÁS:
    autenticación obligatoria
)
```

Esto significa:

- **× ANTES:** `/api/v1/floors`, `/api/v1/rooms`, `/api/v1/desks` eran públicos
- **✓ AHORA:** Necesitan token JWT en header `Authorization: Bearer <token>`

Cómo Adaptar los Controladores

Patrón: Inyectar `Authentication auth`

Todos los controladores deben cambiar de:

```
@GetMapping("/{id}")
public ResponseEntity<FloorResponseDto> getFloorById(@PathVariable Long id)
{
    return
    ResponseEntity.ok(floorMapper.toResponseDto(floorService.getFloorById(id)))
    ;
}
```

A:

```
@GetMapping("/{id}")
public ResponseEntity<FloorResponseDto> getFloorById(
    Authentication auth, // ← AÑADIR ESTO
    @PathVariable Long id) {
    Long userId = (Long) auth.getPrincipal(); // ← EXTRAER userId DEL
    TOKEN
    return
    ResponseEntity.ok(floorMapper.toResponseDto(floorService.getFloorById(id)))
}
```

```
;
}
```

¿Por qué **Authentication auth**?

El **JwtFilter** ahora:

1. Lee el token JWT del header **Authorization: Bearer <token>**
2. Extrae **userId** y **role** del token
3. Crea un **UsernamePasswordAuthenticationToken** con esos datos
4. Lo guarda en **SecurityContextHolder**

Spring inyecta automáticamente ese token en cualquier parámetro de tipo **Authentication** en tu controlador.



Lista de Cambios por Controlador

1. FloorController (/api/v1/floors)

Archivos a cambiar:

- **src/main/java/com/ediae/ecotrack_office/assets/controller/FloorController.java**

Cambios necesarios:

Método	Cambio
getAllFloors()	Añadir Authentication auth (GET público según lógica)
getFloorById()	Añadir Authentication auth
getFloorsByOrganizationId()	Añadir Authentication auth + verificar que el usuario pertenece a esa org
getFloorsStatusByOrganizationId()	Añadir Authentication auth + verificar org
createFloor()	Añadir Authentication auth + verificar permisos ADMIN
updateFloor()	Añadir Authentication auth + verificar permisos ADMIN
deleteFloor()	Añadir Authentication auth + verificar permisos ADMIN

Patrón para cada método:

```
@GetMapping
public ResponseEntity<List<FloorResponseDto>> getAllFloors(Authentication
```

```

auth) {
    Long userId = (Long) auth.getPrincipal();
    System.out.println("GET /floors solicitado por userId: " + userId);

    List<FloorResponseDto> floors = new ArrayList<>();
    floorService.getFloors().forEach(floor ->
    floors.add(floorMapper.toResponseDto(floor)));
    return ResponseEntity.ok(floors);
}

@PostMapping
public ResponseEntity<FloorResponseDto> createFloor(
    Authentication auth,
    @RequestBody FloorRequestDto floorRequestDto) {

    // Verificar que el usuario es ADMIN
    boolean isAdmin = auth.getAuthorities().stream()
        .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
    if (!isAdmin) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    Long userId = (Long) auth.getPrincipal();
    FloorResponseDto floor =
    floorMapper.toResponseDto(floorService.createFloor(floorRequestDto));
    return ResponseEntity.ok(floor);
}

```

2. RoomController (/api/v1/rooms)

Archivos a cambiar:

- `src/main/java/com/ediae/ecotrack_office/assets/controller/RoomController.java`

Mismo patrón que **FloorController**: todos los métodos reciben **Authentication auth**.

Métodos afectados:

- `getAllRooms()` → Añadir **Authentication auth**
- `getRoomById()` → Añadir **Authentication auth**
- `getRoomsByFloorId()` → Añadir **Authentication auth**
- `getRoomsByOrganizationId()` → Añadir **Authentication auth** + verificar org
- `createRoom()` → Añadir **Authentication auth** + verificar ADMIN
- `updateRoom()` → Añadir **Authentication auth** + verificar ADMIN
- `deleteRoom()` → Añadir **Authentication auth** + verificar ADMIN

3. DeskController (/api/v1/desks)

Archivos a cambiar:

- `src/main/java/com/ediae/ecotrack_office/assets/controller/DeskController.java`

Mismo patrón: todos los métodos reciben `Authentication auth`.

Métodos afectados:

- `getAllDesks()` → Añadir `Authentication auth`
- `getDeskById()` → Añadir `Authentication auth`
- `getDesksByRoomId()` → Añadir `Authentication auth`
- `createDesk()` → Añadir `Authentication auth` + verificar ADMIN
- `updateDesk()` → Añadir `Authentication auth` + verificar ADMIN
- `deleteDesk()` → Añadir `Authentication auth` + verificar ADMIN

4. ReservationController (/api/v1/reservations)

Archivos a cambiar:

- `src/main/java/com/ediae/ecotrack_office/reservation/controller/ReservationController.java`

Cambios especiales:

```
@GetMapping("/user/{id}")
public List<ReservationResponseDto> getReservationsByUserId(
    Authentication auth,
    @PathVariable Long id) {
    Long userId = (Long) auth.getPrincipal();

    // Verificar que el usuario solo pueda ver sus propias reservas
    // (a menos que sea ADMIN)
    if (!userId.equals(id)) {
        boolean isAdmin = auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
        if (!isAdmin) {
            // Lanzar excepción o retornar 403
            throw new AccessDeniedException("No puedes ver las reservas de
otro usuario");
        }
    }

    List<ReservationModel> models = service.getReservationsByUserId(id);
    List<ReservationResponseDto> dtos = new ArrayList<>();
    for (ReservationModel model : models) {
        dtos.add(ReservationMapper.toResponseDto(model));
    }
    return dtos;
}

@PostMapping
public ReservationResponseDto createReservation(
```

```

        Authentication auth,
        @RequestBody ReservationCreateDto dto) {
    Long userId = (Long) auth.getPrincipal();

    System.out.println("POST /reservations por userId: " + userId);
    System.out.println("Received DTO: " + dto);

    return ReservationMapper.toResponseDto(service.createReservation(dto));
}

```



Verificación de Permisos: Patrón Reutilizable

Para no repetir código, crea un **Guard util** reutilizable:

```

// archivo: com/ediae/ecotrack_office/shared/guard/AuthGuard.java
@Component
public class AuthGuard {

    public Long getUserIdFromAuth(Authentication auth) {
        return (Long) auth.getPrincipal();
    }

    public boolean hasRole(Authentication auth, String role) {
        return auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_" + role));
    }

    public boolean isAdmin(Authentication auth) {
        return hasRole(auth, "ADMIN");
    }

    public boolean isTechnician(Authentication auth) {
        return hasRole(auth, "TECHNICIAN");
    }
}

```

Luego en los controladores:

```

@RestController
@RequestMapping("/api/v1/floors")
public class FloorController {

    @Autowired
    private AuthGuard authGuard;

    @PostMapping
    public ResponseEntity<FloorResponseDto> createFloor(
        Authentication auth,

```

```
        @RequestBody FloorRequestDto floorRequestDto) {

    if (!authGuard.isAdmin(auth)) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    Long userId = authGuard.getUserIdFromAuth(auth);
    // ...

}
}
```

Cambios en Frontend (Angular)

Antes (con headers X-User-Id)

```
// interceptors/auth.interceptor.ts - ANTES
const cloned = req.clone({
  setHeaders: {
    'X-User-Id': String(CURRENT_USER_ID),
    'X-User-Role': CURRENT_USER_ROLE,
  },
});
```

Ahora (con JWT)

```
// interceptors/auth.interceptor.ts - AHORA
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('jwt_token'); // 0 de donde guardes el JWT

  if (token) {
    const cloned = req.clone({
      setHeaders: {
        'Authorization': `Bearer ${token}`
      },
    },
  );
  return next(cloned);
}

return next(req);
};
```

Checklist de Validación

Después de adaptar cada controlador, verifica:

- ☐ Todos los métodos públicos reciben parámetro `Authentication auth`
- ☐ Se extrae `userId` con `(Long) auth.getPrincipal()`
- ☐ Los métodos de creación/actualización/borrado verifican permisos ADMIN
- ☐ Los métodos de lectura verifican que el usuario tiene acceso a esa organización
- ☐ El frontend envía el token JWT en header `Authorization: Bearer <token>`
- ☐ El token es generado tras login en `/api/v1/auth/login`
- ☐ Los logs muestran el `userId` para debugging



Errores Comunes

Error	Solución
<code>NullPointerException</code> en <code>auth.getPrincipal()</code>	El parámetro <code>Authentication auth</code> no está en la firma del método. Añádalo.
401 Unauthorized en todos los endpoints	El cliente no está enviando el JWT. Verifica que el interceptor Angular está registrado.
403 Forbidden en <code>POST /floors</code>	El usuario no es ADMIN. Verifica los roles en la respuesta de login.
El controlador no recibe <code>Authentication</code>	Asegúrate de que la clase es <code>Authentication</code> de <code>org.springframework.security.core.Authentication</code>



Orden de Implementación

1. **Primero:** Adaptar `UserController` (ya está hecho como referencia)
2. **Segundo:** Adaptar `FloorController`
3. **Tercero:** Adaptar `RoomController`
4. **Cuarto:** Adaptar `DeskController`
5. **Quinto:** Adaptar `ReservationController`
6. **Sexto:** Crear `AuthGuard` utility para reutilizar verificaciones
7. **Séptimo:** Actualizar frontend con JWT interceptor
8. **Octavo:** Testear con Postman: añadir token a Authorization header



Referencias

- **UserController:** Patrón completo ya implementado en `/api/v1/users`
- **SecurityConfig:**
`/src/main/java/com/ediae/ecotrack_office/auth/config/SecurityConfig.java`
- **JwtFilter:** `/src/main/java/com/ediae/ecotrack_office/auth/filter/JwtFilter.java`
- **JwtService:** Lee este archivo para entender cómo extraer `userId` y role del token



Tips

- Usa `System.out.println()` durante el desarrollo para ver qué `userId/role` llega en cada request
- Postman: en la pestaña "Authorization", selecciona "Bearer Token" e introduce tu JWT
- Los equipos que reciban este documento: deben aplicar estos cambios a TODOS los controladores